



Datasets y Herramientas para Audio

Audio: la tercera modalidad — de la teoría al dato

Profesores:

Alain Raymond | **Gabriel Sepulveda**



1 | El audio está en todas partes



¿Para qué sirve el audio en IA?

Se usa en más lugares de los que uno imagina.

Habla

Transcribir call centers,
asistentes de voz,
biometría.

Sonidos del mundo

Detectar una máquina
que va a fallar,
identificar aves,
alarmas.

Música

Shazam,
recomendación,
separar pistas,
generación.



Tres familias de problemas de audio

Casi todo problema cae, al menos, en una de estas tres.

01

Música

Géneros,
transcripción,
separación de
fuentes. *Ej: Shazam.*

02

Habla

Reconocer, sintetizar,
identificar al hablante.
Ej: Siri.

03

Ambiente

Alarmas, máquinas,
fauna, ciudad. *Ej:
detector de fallas.*

En la práctica se mezclan

Música que tapa una voz, una máquina que falla bajo el ruido de la fábrica: el audio real casi nunca viene puro.



¿Dónde se usa esto hoy?

Tres lugares donde el audio ya está trabajando, y cómo funciona en cada uno.

En una planta industrial

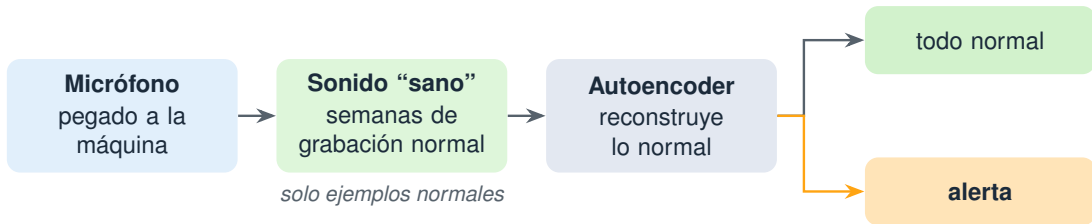
Un rodamiento gastado **cambia el sonido de la máquina antes de romperla**. Se pega un micrófono al equipo, se entrena un modelo con el sonido “sano” y avisa cuando algo se sale del patrón. Se usa en compresores, transformadores y ascensores.

En un banco

La voz funciona como huella: al llamar, el modelo la compara con un patrón guardado y decide si la persona es quien dice ser. Se volvió urgente porque hoy **se clona una voz con pocos segundos de audio**.

Cómo funciona el detector de fallas

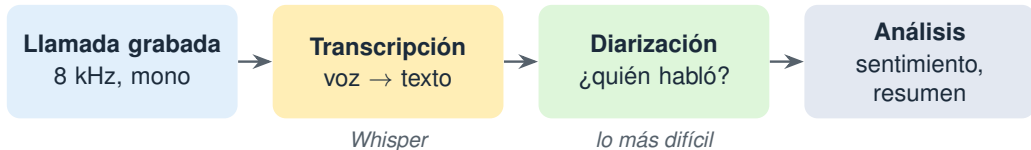
No hace falta saber qué se rompió: basta con notar que dejó de sonar igual.



¿Y por qué no responde "normal" siempre? Porque no es un clasificador: se entrena a **comprimir y reconstruir** el sonido normal (autoencoder). Lo anómalo no calza con lo que aprendió y **se reconstruye mal**; ese error de reconstrucción es la alerta.

El caso: un call center

Miles de llamadas grabadas al día. ¿Qué se puede hacer con ellas?



Por qué a la empresa le importa

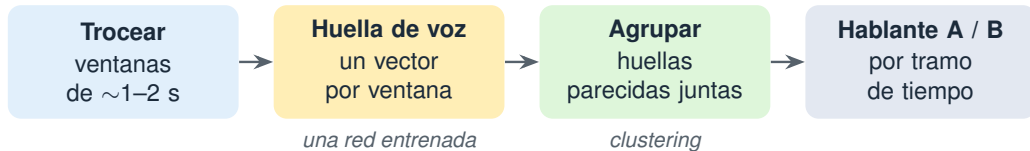
Hoy revisan a mano una muestra chica de llamadas; con esto revisan **todas**. Y el resumen le ahorra al agente 30 a 60 segundos por llamada.

Dato de terreno: el cuello de botella suele ser separar a los hablantes, no medir el sentimiento.

Fuentes: gladia.io (speech-to-text para contact centers) talkdesk.com (speech analytics)

¿Y cómo se sabe quién habló?

Con la misma idea del banco: la voz como huella.



La misma huella que usa el banco para autenticar, usada para *agrupar*: ventanas con huella parecida = misma persona. ¿Por qué cuesta tanto? Porque la gente **se interrumpe** (dos voces en la misma ventana), no se sabe **cuántos** hablantes hay, y el teléfono viene a 8 kHz y en un solo canal.



¿Qué vamos a ver hoy?

Cuatro bloques, y el práctico donde todo se junta.

01

El audio en el disco

De la onda al arreglo:
sample rate, bits,
formatos, ffmpeg.

02

Qué cambia con audio

Nyquist,
espectrogramas, y
del archivo al tensor.

03

Data augmentation

Fabricar datos nuevos
que conserven la
etiqueta.

04

Datasets

Cuáles existen,
dónde conseguirlos,
con qué licencia.

Y en el práctico

Todo junto en Colab: un clasificador de géneros musicales, de los WAV al modelo.

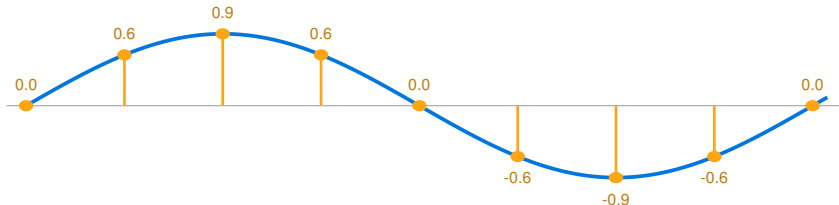


2 | El audio en el disco



El audio es un arreglo de amplitudes

Lo que queda en el disco: la altura de la onda, muchas veces por segundo.



↓ cada punto, en orden, a una casilla

canal 1	0.0	0.6	0.9	0.6	0.0	-0.6	-0.9	-0.6	0.0
canal 2	.	.	.	.	.	.	.	.	.

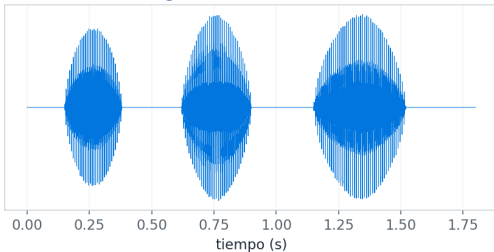
= (2, 9)

Estéreo = 2 filas, como los canales de una imagen RGB. Es exactamente lo que entrega `prchaudio.load()` en el lab.

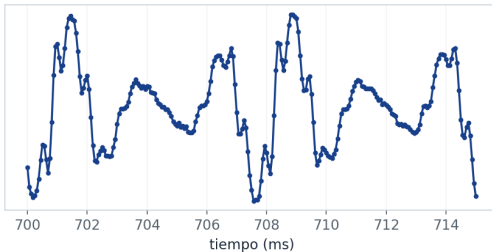
¿Cómo se ve el audio por dentro?

Lo que parece un bloque sólido son miles de números por segundo.

1,8 segundos = 28.800 números



Zoom a 15 ms: acá está la onda de verdad



Cada punto del zoom es una muestra. Por eso un segundo de calidad CD son 44.100 números por canal.



¿Cuánto pesa? Sample rate y bit depth

Dos números deciden calidad y tamaño.

Sample rate (Hz)

Muestras por segundo. CD = 44.100
Hz. Teléfono = 8.000 Hz.

Bit depth

Bits por muestra (precisión). 16 bits =
65.536 niveles de amplitud.

$$\text{tamaño}_{\text{sin comprimir}} = \text{sr} \times \text{bits} \times \text{canales} \times \text{duración}$$

Ejemplo

3 min, estéreo, 44.1 kHz, 16 bits $\approx$ **31 MB**. El mismo audio en MP3 $\approx$ 3 MB.

esto es la Actividad 2 del lab

el call center Una llamada pesa $\approx$ 1 MB por minuto (8 kHz, 16 bits, mono): por eso se guardan comprimidas.



Formatos: ¿comprimir o no?

El mismo sonido, guardado de tres maneras distintas.

01

Sin comprimir

WAV, AIFF, PCM.

Pesado, exacto. El
“BMP” del audio.

02

Lossless

FLAC, ALAC.

Comprime *sin* perder
nada (~40%).

03

Lossy

MP3, AAC, OGG.

Liviano, descarta lo
inaudible.

Para tener presente

Lo que un formato lossy bota **no vuelve**: el original conviene guardarlo sin pérdida. ¿Y cuándo molesta esa pérdida? Depende de la tarea →

¿Y cuándo importa la pérdida?

Depende de la tarea. MP3 está optimizado para el oído humano: bota lo que no escuchamos.

MP3 basta y sobra

Clasificar habla o música, transcribir llamadas (ya vienen comprimidas), géneros musicales. Si la tarea es *perceptual*, lo que MP3 botó tampoco lo necesitaba el modelo.

Mejor sin pérdida

Fallas de máquinas con firmas sobre ~16 kHz, bioacústica y ultrasonido, sonidos cardíacos o pulmonares, audio forense, síntesis de alta fidelidad.

La pregunta que ordena todo: ¿la señal que me importa vive donde el oído humano escucha? Si sí, lossy sirve. Si no, lossy puede botar justo lo que buscabas.



¿Y cómo consigo el WAV?

El audio real casi nunca llega en el formato que necesitan.

Así llega en la práctica

Reuniones grabadas en .mp4, audios de WhatsApp en .opus, streaming en .aac, notas del celular en .m4a. Torchaudio espera formatos de audio como .wav o .mp3.

ffmpeg convierte todo

Una herramienta de consola, gratis, que lee prácticamente cualquier formato de audio o video y lo deja donde quieran. Es el paso 0 de casi todo proyecto de audio real.

Dos líneas que resuelven el 90%

```
ffmpeg -i reunion.mp4 audio.wav
```

```
ffmpeg -i nota.opus -ar 16000 -ac 1 nota.wav
```

saca el audio de un video
y de paso resamplea y deja mono

lo practican en el anexo del lab



¿Comprimir o no el dataset?

Para entrenar, depende de si el cuello de botella es el disco o la CPU.

Sin comprimir (WAV)

- + Carga sin CPU: no frena el *dataloader*.
- + Exacto y determinista.
- Pesa muchísimo: storage caro.
- Más I/O por época.

Comprimido (FLAC / MP3)

- + Mucho menos disco y transferencia.
- + Si el cuello es el I/O, más rápido.
- Decodificar en cada carga: posible cuello de botella.
- *Lossy*: el codec puede volverse un **sesgo**.

En la práctica

Decodificar es barato ($\sim 1.000\times$ tiempo real): el codec casi nunca es el cuello.

Los datasets grandes se distribuyen **comprimidos** y se decodifican al vuelo:

- 7 LibriSpeech en FLAC, GigaSpeech en Opus, Common Voice en MP3. La decisión real es la anterior: ¿la tarea tolera pérdida?



3 | Qué cambia al trabajar con audio



Audio, la tercera modalidad

Ya trabajaron con texto e imágenes. El audio toma cosas de los dos.

	Texto	Imagen	Audio
Forma del dato	tokens en secuencia	grilla 2D de píxeles	señal 1D en el tiempo
Tamaño típico	corto	medio	enorme ($\sim$ miles/seg)
¿Secuencial?	sí	no	sí
Pre-entrenado	un LLM	ResNet / ViT	Whisper / wav2vec

La idea clave

No hay una representación única: el audio se trabaja a veces **crudo**, a veces como **espectrograma** (y ahí es una imagen), como **MFCC** o como **embeddings** preentrenados. Elegir la representación es la primera decisión del pipeline.

Qué cambia: tres frentes

Primero la señal, después la representación, al final la preparación.

01

El muestreo

La señal es continua:
cada cuánto medirla
y qué se pierde si
mido lento (Nyquist,
aliasing).

02

La representación

Crudo,
espectrograma,
MFCC o embeddings:
las transforms y sus
parámetros.

03

La preparación

Detalles prácticos
antes de entrenar, y
el lado humano de
los datos.

el call center

En el call center: las llamadas llegan a 8 kHz (muestreo), se vuelven features (representación) y hay que poder juntarlas en batches (preparación).



El sample rate: ¿cada cuánto muestrear?

¿Cuántas fotos por segundo necesito para no perderme nada?

La regla

Para capturar una frecuencia hay que muestrear al **doble** de rápido (al menos 2 muestras por ciclo). Ese límite, la mitad del sample rate, se llama **frecuencia de Nyquist**.

La analogía

Es la rueda que en los videos parece girar al revés: la cámara (el muestreo) es demasiado lenta para seguir el giro real, y el cerebro “ve” otro movimiento. Con el sonido pasa igual.



¿Y si elijo mal el sample rate?

Dos cosas que es facilísimo confundir, y no son lo mismo.

(a) Reproducir a otra tasa

Se reproducen las mismas muestras más rápido o más lento: la voz suena aguda o grave. **No se pierde información.**

es lo que ve el Colab

(b) Muestrear bajo Nyquist

Si se graba por debajo del doble de la frecuencia más alta, esa frecuencia se *disfraza* de una más baja. **Se pierde para siempre.**

el call center A 8 kHz, Nyquist queda en 4.000 Hz: la “s” pierde sus agudos y suena como “f”. Por eso deletreamos por teléfono.



La representación: las transforms

`torchaudio.load()` da el tensor; las transforms lo convierten en features.

Los parámetros clave

`win_length`: tamaño de la ventana

`hop_length`: cada cuánto se avanza

`n_mels`: cuántas bandas de frecuencia

La analogía que ya conocen

La ventana es como el **kernel** de una convolución; el hop, como el **stride**.

¡Misma intuición de la clase de CNNs!

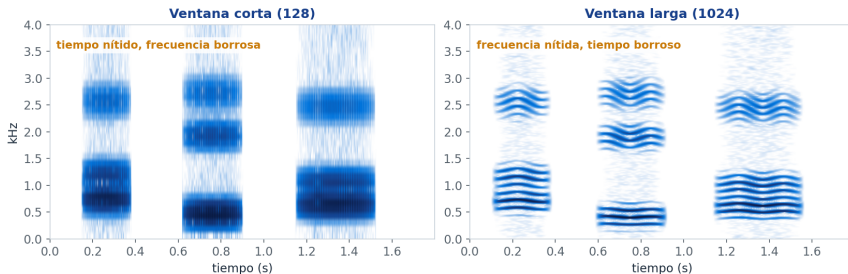
Probémoslo

Suban la ventana y miren el trade-off: ventana grande = mejor resolución en frecuencia, peor en tiempo.

Link / QR: espectrograma-ventana-hop

¿Y qué ventana uso?

Depende de qué necesita distinguir el modelo. El mismo audio, dos ventanas:



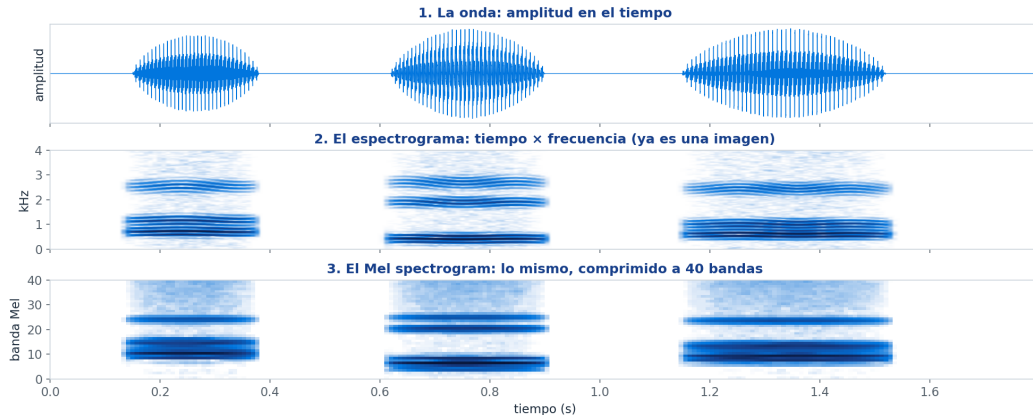
La decisión

Ventana **corta** si importa el *cuándo*: golpes, consonantes, inicios de eventos.

Ventana **larga** si hay que separar frecuencias parecidas: notas, armónicos, tonos de voz. En habla lo típico es ~ 25 ms; en música, ventanas más largas. Si no hay razones, partan del default y *miren* el espectrograma.

El mismo sonido, de tres formas

Onda, espectrograma y Mel: tres representaciones del mismo audio.Cuál entra al modelo depende de la tarea.





El tensor que entra al modelo

Calcular sus dimensiones es como calcular el output de una convolución.

$$\text{frames} = \left\lfloor \frac{N - \text{win_length}}{\text{hop_length}} \right\rfloor + 1 \quad \Rightarrow \quad (\text{canales}, \text{n_mels}, \text{frames})$$

Lo que sale

Una matriz 2D por canal: un eje es frecuencia (n_mels), el otro es tiempo (frames).

Actividad 3 del lab

El puente

Si es una imagen...

→ la **próxima clase** le tiramos una **CNN**.

el call center Una llamada de 5 min: (1, 2.400.000) muestras crudas; tras un Mel de 64 bandas queda (1, 64, ~4.700). De archivo gigante a imagen razonable.



Antes de entrenar: detalles que conviene cuidar

Errores comunes al preparar audio, fáciles de evitar si los conocen.

Resamplear no crea información

Subir el sample rate solo interpola: como hacer zoom a una foto, no aparece detalle nuevo.

Largo variable → batching

El audio no viene de tamaño fijo.
PyTorch no arma el batch: se arregla con `collate_fn`. **lo viven en el lab**

Mono vs estéreo

¿Promediar los canales o no? A veces se pierde información espacial que importaba.

Normalización y silencios

Conviene normalizar amplitudes y recortar silencios (VAD). Las etiquetas pueden no cubrir todo el audio.



La parte humana

En el call center esto no es opcional.

Consentimiento y privacidad

Grabar llamadas exige consentimiento.
La voz es **dato biométrico** y cada llamada tiene datos personales (PII).

Sesgo

¿El dataset cubre acentos, géneros e idiomas? Un ASR entrenado solo en inglés “limpio” falla con los clientes reales.

Copyright

En música casi todo tiene derechos.
Por eso lo abierto suele ser Creative Commons o instrumental.

Buena práctica

Conviene anonimizar, minimizar y documentar de dónde vienen los datos: ahorra problemas legales y mejora el modelo.



4 | Data augmentation



Cuando faltan datos, se fabrican

Variantes que suenan distinto pero conservan la etiqueta.

La idea

Un rock 10% más rápido sigue siendo rock. Con variantes, el modelo debe aprender **qué hace que algo sea rock** en vez de memorizar cada grabación. En el lab lo miden: la brecha train-test se cierra.

El criterio, antes de escribir código

¿La transformación es una **invariancia real** del problema? ± 4 semitonos vale para clasificar género; destruye la tarea si es detectar tonalidad o identificar al hablante.

el call center ¿Una llamada más rápida sigue siendo el mismo reclamo? Sí. ¿La misma voz para la biometría del banco? No.



La relación señal-ruido (SNR)

Un cociente de potencias, medido en decibeles.

$$\text{SNR} = 10 \log_{10} \frac{P_{\text{señal}}}{P_{\text{ruido}}} [\text{dB}] \quad P = \text{promedio de amplitud}^2$$

SNR	Qué significa
0 dB	señal y ruido con la misma potencia
+10 dB	la señal tiene 10 veces la potencia del ruido
+20 dB	100 veces (cada 10 dB es un factor 10)
-10 dB	el ruido es 10 veces más potente que la señal

La imagen mental

31 Conversar en una fiesta: la voz es la señal, el murmullo es el ruido. En una pieza callada el SNR es alto; al lado del parlante es bajo, o negativo.



Sumar ruido: la perilla es el SNR

La augmentation más directa: otra señal de fondo. La etiqueta no cambia.

Calibrar el oído

La cantidad de ruido se fija por SNR, no por amplitud.

20 dB: el ruido apenas se percibe

10 dB: fondo claramente audible

3 dB: casi tan fuerte como la señal

En el lab escuchan los tres.

Blanco o real

El blanco ya regulariza. Mejor aún: **grabaciones del ruido real** donde va a operar el modelo — que el ruido de train se parezca al de producción.

`Fa.add_noise(señal, ruido, snr)` escala el ruido al SNR pedido.

En entrenamiento

El SNR se **sortea en un rango** (por ejemplo, 10 a 20 dB): cada época suena distinto. **sección 6.4 del lab**

¿Y en datos reales? El SNR se estima: silencios (VAD), WADA-SNR, o SQUIM (`torchaudio.pipelines`). • [Tutorial de augmentation](#).



El otro ruido: tapar en vez de sumar

SpecAugment: hacer cero componentes del espectrograma, al azar.

La receta

Sobre el espectrograma se sortean **bandas de frecuencia** (filas) y **tramos de tiempo** (columnas), y se hacen **cero**. Si conocen Cutout en imágenes, es exactamente eso.

Por qué funciona

Es **dropout con estructura**: se apagan regiones contiguas, no unidades sueltas. Si el modelo dependía de una sola banda, taparla entera lo obliga a buscar evidencia en otra parte.

Y es casi gratis

Opera sobre el espectrograma que ya está en la GPU: dos líneas en el loop de entrenamiento. **sección 6.3 del lab**



Cambiar duración o tono, por separado

Reproducir más rápido cambia los dos a la vez; estas herramientas mueven uno solo.

`time_stretch`

Más lento o más rápido, **mismo tono**.
Un rock 10% más rápido sigue siendo rock.

`pitch_shift`

Más agudo o más grave, **misma duración**. ± 2 semitonos no cambian el género.

En voz, ojo: los formantes se corren y suena a otra persona.

Se puede, pero es caro

Separar duración y tono requiere un algoritmo no trivial (phase vocoder; no entramos al detalle) que cuesta ~ 1 s por archivo. Por eso no va en el dataloader: se **precalcula una vez**, offline. Las de ruido, en cambio, son casi gratis — por eso son la primera línea.



Dónde va cada una

El costo decide el lugar en el pipeline.

Costo	Operaciones	Dónde va
Barato, en CPU	recorte aleatorio, ganancia, ruido (SNR)	en el <code>collate_fn</code>
Gratis, en GPU	SpecAugment (el Cutout del espectrograma)	en el loop de entrenamiento
Caro (~ 1 s por archivo)	<code>pitch_shift</code> , <code>time_stretch</code>	offline , precalculado

Dos reglas

Augmentation **solo en train** (el test se deja quieto para que la métrica sea comparable) y **distinta en cada época** (esa es la gracia: el modelo nunca ve dos veces el mismo ejemplo). **el collate de la Parte 2 del lab**



5 | Datasets



Hay datasets de tres tamaños

El salto de los últimos años no fue de algoritmos: fue de escala de datos.

01

Didácticos

GTZAN,
UrbanSound8K,
ESC-50. Para
aprender. *Es lo del
lab.*

02

Benchmarks

LibriSpeech,
AudioSet, FSD50K.
Estándares de
investigación.

03

Escala web

Common Voice,
GigaSpeech, Emilia.
Cientos de miles de
horas.

Para dimensionar

Whisper no se entrenó con ninguno de estos: usó ~**680.000 horas** de audio web débilmente etiquetado. Ahí está la frontera hoy.

Radford et al. (2022), *Whisper*: pares (audio, transcripción) encontrados en la web, filtrados, no verificados por humanos. En large-v3 la escala subió a ~1M de horas débilmente etiquetadas + 4M pseudo-etiquetadas por el modelo anterior.



Cómo elegir un dataset

Más que los nombres, estos cuatro ejes deciden si les sirve.

Escala vs. limpieza

Etiquetado a mano (limpio, chico) vs. web débilmente etiquetado (sucio, gigante). La frontera se fue por la escala.

Disponibilidad

¿Les dan el audio (FSD50K) o solo un link de YouTube que se cae (AudioSet)?

Licencia y privacidad

Lo mejor suele ser pagado (LDC) o con copyright. La voz es dato personal.

Representación / sesgo

¿Qué idiomas, acentos y géneros están, y cuáles no?

el call center

No existe un dataset público de llamadas chilenas: el flujo real es

38 pre-entrenar con datos públicos y fine-tune con las llamadas propias.



Datasets chicos para empezar

Todos son gratis y se bajan rápido. Alcanzan para un proyecto sin GPU.

Dataset	Qué trae	Dónde
ESC-50	2.000 clips de 5 s, 50 clases de sonidos ambientales	github.com/karolpiczak/ESC-50
UrbanSound8K	8.732 clips de hasta 4 s, 10 clases urbanas, en 10 folds	urbansounddataset.weebly.com
Speech Commands	65.000 clips de 1 s, 30 palabras cortas	tensorflow.org/datasets
GTZAN	1.000 temas de 30 s, 10 géneros musicales	torchaudio.datasets.GTZAN

Cómo se bajan

Varios vienen empaquetados en `torchaudio.datasets` y en Hugging Face: se

39 bajan con una línea. **GTZAN es el del lab**



Los datasets grandes

Los que se usan en investigación y en producción.

Dataset	Qué trae	Dónde
LibriSpeech	1.000 h de lectura en inglés, 16 kHz, licencia libre	openslr.org/12
FSD50K	51.000 clips, 200 clases, a mano. Entrega el audio	zenodo.org/records/4060432
AudioSet	2M de clips, 500+ clases. Solo IDs de YouTube	research.google.com/audioset
Common Voice	Multilingüe, con español y metadata demográfica	commonvoice.mozilla.org
GigaSpeech	10.000 h de inglés transcrito: podcasts, YouTube	github.com/SpeechColab
Emilia	101.000 h en 6 idiomas. Para generar voz (TTS)	huggingface.co/amphion/Emilia

Antes de descargar

No todo es un click: GigaSpeech pide **formulario y aprobación**, y tanto él como Emilia son **solo para uso no comercial** (la variante Emilia-YODAS sí permite uso comercial). Revisen siempre licencia y forma de acceso.

No todas las etiquetas son iguales

Qué tan “fina” es la anotación cambia por completo lo que se puede hacer.

01

Por clip

“Esto es jazz”. Una etiqueta para todo el archivo. *Ej: GTZAN.*

02

Débil / multi

Varias etiquetas, sin decir cuándo. *Ej: AudioSet.*

03

Alineada

Cada evento ubicado en el tiempo. *Ej: MusicNet, LibriSpeech.*

Detalle del lab

En MusicNet el audio termina en silencio sin anotaciones, y está bien. Saber leer la metadata (qué cubre y qué no) es la mitad del trabajo.



Embeddings preentrenados: wav2vec 2.0

Igual que en texto e imágenes, hay modelos preentrenados listos para usar.
En audio: wav2vec.

Cómo aprende

Se entrena con **miles de horas sin etiquetar**: enmascara trozos del audio y aprende a adivinarlos — la misma idea de BERT, sobre sonido. Nadie etiquetó nada.

Qué entrega

Un vector de 768 dimensiones por cada ~20 ms: un reemplazo *aprendido* de los MFCC. Encima basta un clasificador chico y pocos datos etiquetados propios.

Listo para usar, gratis

En torchaudio.pipelines: WAV2VEC2_BASE, XLSR (multilingüe, 53–128 idiomas) y variantes ya afinadas para transcribir — incluso en **español** (VOXPOPULI_ASR_BASE_10K_ES): transcribir llamadas sin entrenar nada.

anexo del lab